项目报告

项目报告：基于 Qt 的双人五子棋游戏

1. 项目简介

这次课程作业我选择做一个双人对战的五子棋小游戏。之所以选这个题目，一方面是因为五子棋规则比较明确，适合用 C++ 把基本的数据结构、判断逻辑和界面交互串起来；另一方面，它看起来不复杂，但真正做起来会涉及棋盘绘制、鼠标点击定位、胜负判断、悔棋、计时这些内容，比较适合作为一个完整的小项目。
这个项目使用 Qt 6 和 C++ 开发，完成的是本地双人对战版本，不涉及联网和 AI。最后实现的功能包括：基础落子、胜负判定、悔棋、倒计时、超时判负、简化禁手，以及鼠标悬停时的预落子提示。

2. 文件结构、开发环境与项目架构

2.1 开发环境

- 开发语言：C++
- 图形界面库：Qt 6 Widgets
- 构建方式：qmake
- 开发工具：Qt Creator
- 运行平台：macOS

2.2 文件结构

项目主要文件如下：

- `main.cpp`
- 程序入口，负责启动 Qt 应用和主窗口。
- `mainwindow.h / mainwindow.cpp / mainwindow.ui`
- 主窗口部分，负责按钮、信息显示、弹窗提示，以及和游戏逻辑类进行连接。
- `boardwidget.h / boardwidget.cpp`
- 自定义棋盘控件，负责棋盘绘制、棋子绘制、鼠标点击和鼠标悬停效果。
- `gomokugame.h / gomokugame.cpp`
- 游戏核心逻辑，负责棋盘数据、落子规则、胜负判定、悔棋、倒计时和禁手处理。
- `docs/`
- 用于保存开发进度、设计说明、测试记录和项目报告。

2.3 采用的架构

这个项目采用的是比较简单但很实用的“界面层 + 逻辑层”分离方式。

- `MainWindow` 负责界面控制和信息展示。
- `BoardWidget` 负责棋盘显示和鼠标交互。
- `GomokuGame` 负责真正的游戏规则和数据处理。

这样做的好处是，界面代码和规则代码不会混在一起。比如以后如果想改界面样式，主要改 `MainWindow` 和 `BoardWidget`；如果想改规则，比如加入新的禁手判定，就主要改 `GomokuGame`。整体会更清晰，也更符合 C++ 面向对象编程的思路。

在具体实现上，这个架构还有几个比较明确的约定：

- 棋盘固定为 `15 x 15` 交点，也就是数组下标范围 `0 ~ 14`。
- `BoardWidget` 的最小尺寸设置为 `520 x 520`，保证窗口初始打开时棋盘不会太小。
- `MainWindow` 不直接保存棋盘数据，它只负责把按钮点击、棋盘点击这些界面操作转发给 `GomokuGame`。
- `GomokuGame` 不负责绘图，它只提供“这个位置是什么棋子”“当前是谁走”“游戏是否结束”这些结果给界面层读取。

3. 开发过程：从最开始的想法到后面的不断完善

一开始我想做的其实是一个最基础的五子棋程序，目标很简单，就是能下棋、能判断谁赢了、能重新开始。最初的想法是先把最核心的功能做出来，不去追求太复杂的扩展，不然容易把项目做乱。

最开始实现的时候，我先把整体拆成三个部分：主窗口、棋盘控件和游戏逻辑。这样做是因为如果一开始就把所有代码都写在主窗口里，后面越写越容易混乱。先把结构搭好，后面加功能会顺很多。

第一步完成的是基础对战功能。这个阶段主要解决的是棋盘怎么画、鼠标点下去怎么知道是落在哪个交点、以及落子后如何保存棋盘状态。等这些完成以后，程序已经可以实现黑白双方轮流落子了。

接下来做的是胜负判定。刚开始只是想实现“谁先连成五子谁赢”，这是最基本也是最关键的部分。这个功能完成后，整个游戏就已经能正常玩起来了。

在基础功能完成之后，我开始继续往上补细节。比如加入“悔棋”，这样对局时不会因为点错一步就只能重开；再比如加入计时功能，本来最开始做的是累计用时显示，后来觉得倒计时更像正式一点的对局，所以又改成了双方倒计时，并且支持超时判负。

后面又补了两个比较有展示效果的功能。一个是鼠标悬停时的预落子提示，这样用户在落子前能更直观看到当前位置；另一个是简化禁手，先做了黑棋长连判负。这样既能体现规则扩展，也不会把项目复杂度一下拉得太高。

整个开发过程中，项目不是一次性想完整再去写，而是先做出一个能运行的基本版本，再一边测试一边补功能、一边修问题。比如后面测试时又发现胜利时会弹出两个提示框，这种问题就是在不断运行和调整中发现并解决的。

4. 项目中比较难的部分，以及我是怎么解决的

这个项目真正难的地方，不是界面摆出来，而是几个细节逻辑：点击定位、胜负判断、倒计时处理、禁手判定，还有界面刷新时可能出现的重复弹窗问题。

4.1 鼠标点击如何准确落在交点上

棋盘不是一个按钮矩阵，而是自己画出来的，所以点击之后程序并不知道用户点的是哪一个格点，需要自己去算。
这里我不是凭感觉去画，而是按固定公式来算。
首先，棋盘控件 `BoardWidget` 的最小大小是 `520 x 520` 。在真正绘制棋盘时，我没有直接把整个控件都用满，而是先在四周留出 `12` 像素边距：
也就是 `margin = 12` ，实际可用于放置棋盘背景的矩形区域相当于 `rect().adjusted(12, 12, -12, -12)` 。
然后取这个区域中较短的一边作为棋盘背景的边长，也就是说棋盘背景始终是一个正方形：
这里可以理解为 `side = min(width, height)` 。

在最小尺寸 `520 x 520` 的情况下，实际棋盘背景边长就是 `520 - 12 x 2 = 496` 像素。
接下来计算格子间距。程序里不是直接除以 14，而是除以 `n + 2` ，其中 `n = BoardSize - 1 = 14` ，所以实际除数是 `16` ：
也就是 `cell = side / (n + 2)` 。
也就是说，在最小尺寸下， `cell = 496 / 16 = 31` 像素

这样设计以后，真正的 15x15 交点网格并不是贴在棋盘背景边缘，而是四周再各留出一个 `cell` 的视觉留白。于是第一个交点坐标是：
`origin = (rect.left() + cell, rect.top() + cell)` 。
最后一个交点坐标就是 `origin + 14 x cell` 。

所以在最小尺寸下，14 个间隔总长度是 `14 x 31 = 434` 像素。
两边各留 `31` 像素，看起来会比直接顶到边缘更自然。

鼠标点击转换为格点坐标时，我的处理顺序是：

1. 先判断点击位置是否落在棋盘背景矩形 `boardRect()` 内。
2. 用下面的公式把像素位置换算到以左上角交点为原点的网格坐标：
   `fx = (pos.x() - origin.x()) / cell` ， `fy = (pos.y() - origin.y()) / cell` 。
3. 对 `fx` 和 `fy` 使用 `qRound` 四舍五入，得到最近的交点编号 `ix` 和 `iy` 。
4. 再检查 `ix` 、 `iy` 是否还在 `0 ~ 14` 范围内。
5. 最后再做一次距离判断，避免“点在格子中间也被算作落子”。程序中使用的是半径 `0.5 * cell` 的圆形判定区域：

先计算 `dist2 = dx x dx + dy x dy` ，再判断 `dist2` 是否大于 `(0.5 x cell)^2` 。如果大于这个范围，就认为鼠标离交点太远，不允许落子。

按最小尺寸算，这个允许点击的半径约为 `0.5 x 31 = 15.5` 像素。
也就是说，用户只要点在某个交点附近大约 15 像素以内，就会被识别成这个位置；如果偏得太远，就不会落子。这样既不会要求点得太死，也不会误判。

4.2 胜负判定怎么做

胜负判定是整个项目最核心的算法部分。我的想法不是每下一步都把整个 15x15 棋盘全部扫一遍，而是只围绕“最后一次落子的位置”来判断，因为真正会改变胜负结果的就是刚下的这一步。

具体做法是检查四个方向：

- 水平方向 `(1, 0)`
- 垂直方向 `(0, 1)`
- 主对角线方向 `(1, 1)`
- 副对角线方向 `(1, -1)`

对于每个方向，都从当前落子点开始，分别向正方向和反方向继续找相同颜色的棋子。比如方向是 `(1, 0)`，那就表示先向右数，再向左数；方向是 `(1, 1)`，就表示先沿主对角线右下方向数，再沿左上方向数。
整体公式可以写成： `count = 1 + 正方向连续数量 + 反方向连续数量` 。

这里的 `1` 表示当前刚刚落下的这颗棋子本身。程序里 `countLine()` 的逻辑也很直接：只要当前位置没有越界，并且这个位置的棋子颜色和当前玩家相同，就继续向这个方向移动并累计数量。

如果某个方向的 `count >= 5` ，就说明这一方已经获胜。因为每次只检查最后一手附近的 4 个方向，所以这部分判断的复杂度其实很低，最多就是沿着 4 条线向两边扩展，不需要把整个棋盘重新扫描一遍。

这种做法的优点是逻辑清楚，而且效率也够用。对于课程作业来说，这种实现方式既不复杂，又能体现出算法思路。

4.3 倒计时和超时判负怎么实现

计时一开始我做的是累计用时，但后来觉得倒计时更适合展示，所以又改成了双方各有固定时间，比如 5 分钟。

这里的关键不是“显示时间”，而是“什么时候扣时间、怎么判断超时”。

项目里每一方的初始时间是固定的 `5 * 60 * 1000` 毫秒，也就是 5 分钟。程序中用两个变量分别保存：

- `blackRemainingMs_`
- `whiteRemainingMs_`

当前轮到谁走，`turnTimer_` 就从这一刻开始计时。主窗口里还有一个 `QTimer`，刷新间隔设置为 `200ms`：

也就是每 `200` 毫秒刷新一次界面。

它的作用不是直接扣时间，而是每隔 200 毫秒刷新一次界面，并调用 `game_.tick()` 检查当前方是不是已经超时。

显示剩余时间时，采用的是“基础剩余时间 - 当前回合已经过去的时间”：

如果当前游戏还在进行中，并且正好轮到这一方走棋，那么显示出来的时间就是 `base - turnTimer_.elapsed()`，同时用 `qMax(0, ...)` 保证不会显示成负数。

当一方真正落子时，程序会读取这一回合已经过去的时间 `consumed = turnTimer_.elapsed()`，然后再把它从当前方的剩余时间里扣掉：

黑方对应的是 `blackRemainingMs_ = max(0, blackRemainingMs_ - consumed)`，白方对应的是 `whiteRemainingMs_ = max(0, whiteRemainingMs_ - consumed)`。

如果在刷新阶段发现当前走棋方的剩余时间已经小于等于 0，就直接结束对局，并把胜负状态改成对方获胜：

例如当前轮到黑方，但黑方剩余时间已经小于等于 0，那么程序就会把黑方时间置为 `0`，并把状态改成 `WhiteWin`；如果是白方超时，则相反处理。

这样处理的好处是：

- 倒计时显示是连续变化的，不会卡到“落子时才跳一下”；
- 胜负状态和时间数据是统一维护的，不会出现界面显示结束但逻辑没结束的情况。

另外，悔棋时不能只撤销棋子，还要把这一步消耗掉的时间一起补回去，所以我把每一步的耗时也存进了 `Move` 结构中。这样撤销时不只是“回退棋盘”，连时间状态也能一起回退。

4.4 简化禁手怎么处理

完整的五子棋禁手规则其实比较复杂，比如三三禁手、四四禁手等。如果全部做出来，代码量会明显增加，也容易超出这次作业的合适范围。

所以这次我先做了一个简化版：只处理黑棋长连。

思路其实和胜负判断很接近，也是统计某个方向的连续同色棋子数。不同点在于程序里会先算出当前落子在四个方向里出现的最大连续长度 `maxCount`：

也就是每算完一个方向的 `count`，就更新一次 `maxCount = max(maxCount, count)`。

然后按下面的顺序判断：

- 如果是普通连五，判当前方胜利；
- 如果是黑棋并且某个方向连续数量达到 6 或以上，就不算胜利，而是判定为禁手，直接判黑负。

在代码顺序上，我把“黑棋长连禁手”的判断放在“普通连五胜利”之前：

也就是说，程序会先判断“如果当前玩家是黑棋，并且 `maxCount >= 6`，那么直接判定白方胜利，并把结束原因记为禁手”；只有在没有触发这个条件时，才继续判断普通的 `maxCount >= 5` 获胜条件。

这样可以保证黑棋如果连成 6 个，不会先被当成胜利，再被改成禁手，而是直接按禁手处理，逻辑会更明确。

这种处理方式比较适合课程作业：既体现了规则扩展，也不会一下子把问题做得太复杂。

4.5 为什么会出现两个胜利弹窗

这个问题是在后期测试时发现的。按理说一方胜利后只应该弹出一次提示框，但实际运行时有时会连续弹出两个。

后来检查发现，原因不是胜负判断错了，而是界面刷新定时器和弹窗的事件循环叠在了一起。主窗口每 `200ms` 会刷新一次，而 `QMessageBox::information()` 在显示期间本身也会进入一个事件循环。原来代码的顺序是：

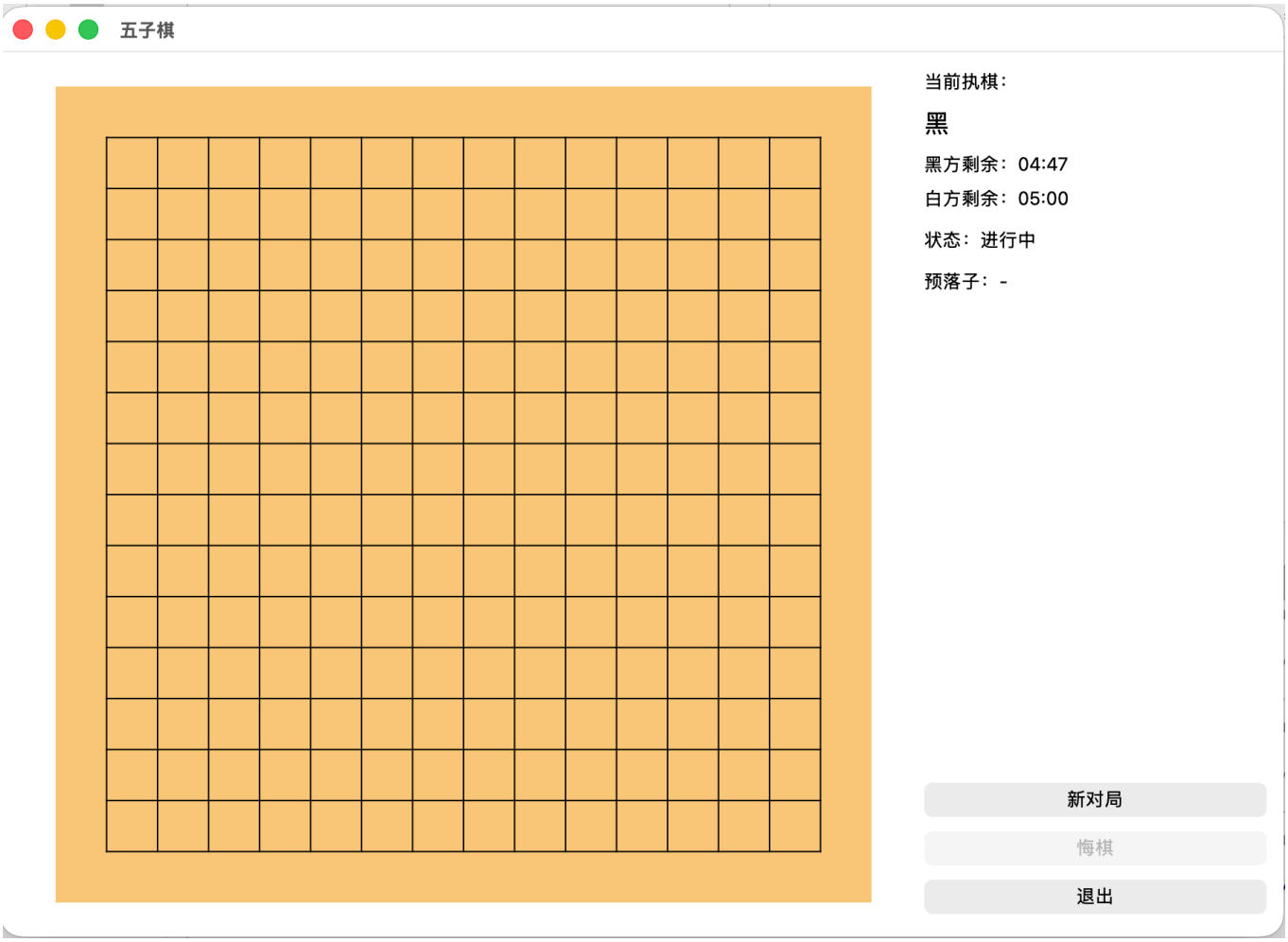
1. 先调用 `QMessageBox::information(...)`
2. 再执行 `gameOverShown_ = true`

问题就出在这两步之间。因为弹窗显示的时候，定时器还可能继续触发 `refreshUi()`，于是程序又再次进入 `showGameOverIfNeeded()`，此时 `gameOverShown_` 还没来得及变成 `true`，所以又弹出一次。

最后的解决方法很直接：先把“已弹窗”标记设为 `true`，再显示提示框。这样即使定时器继续触发，也不会再重复弹出。

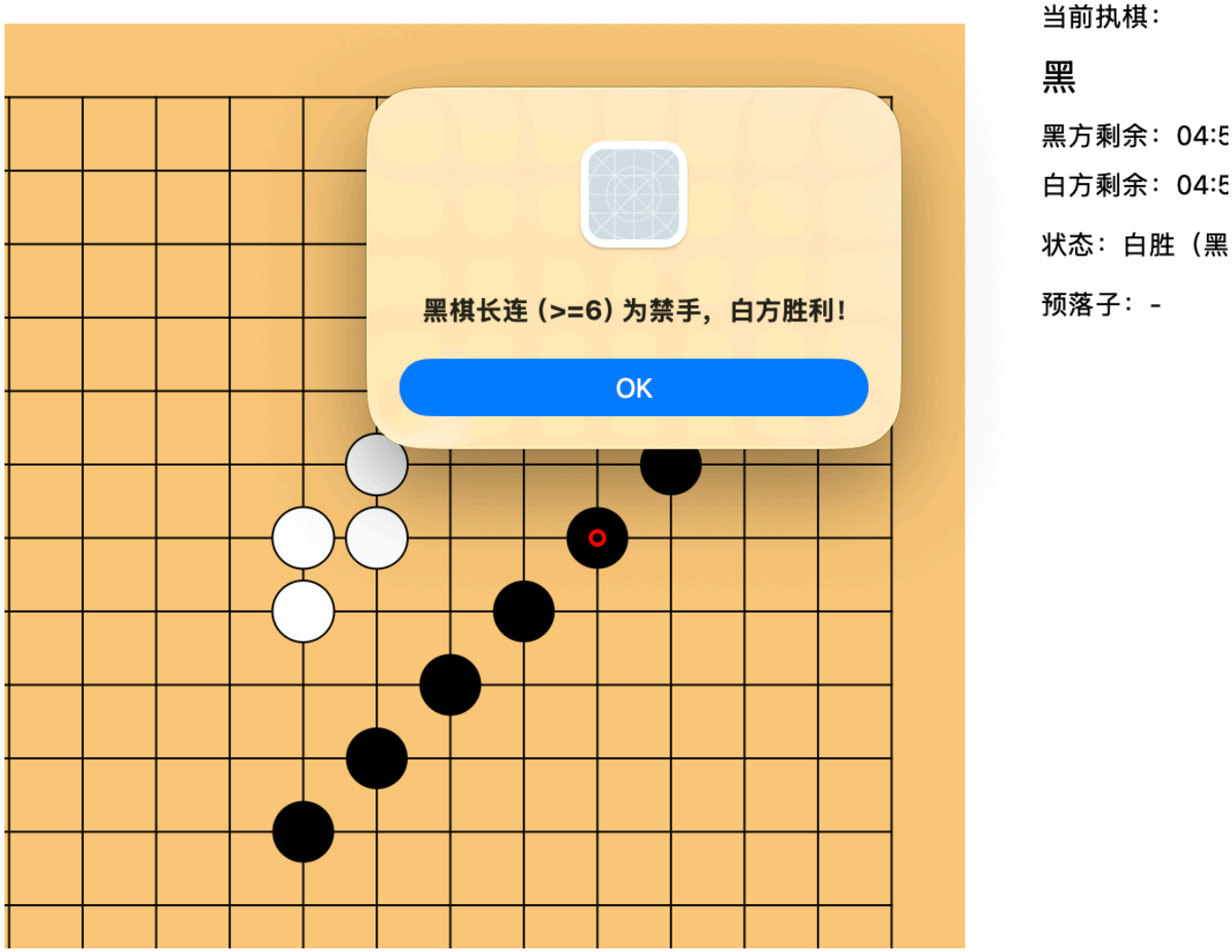
这个问题虽然不算复杂算法，但很能体现项目调试过程中的细节处理。

5.1 对局主界面图

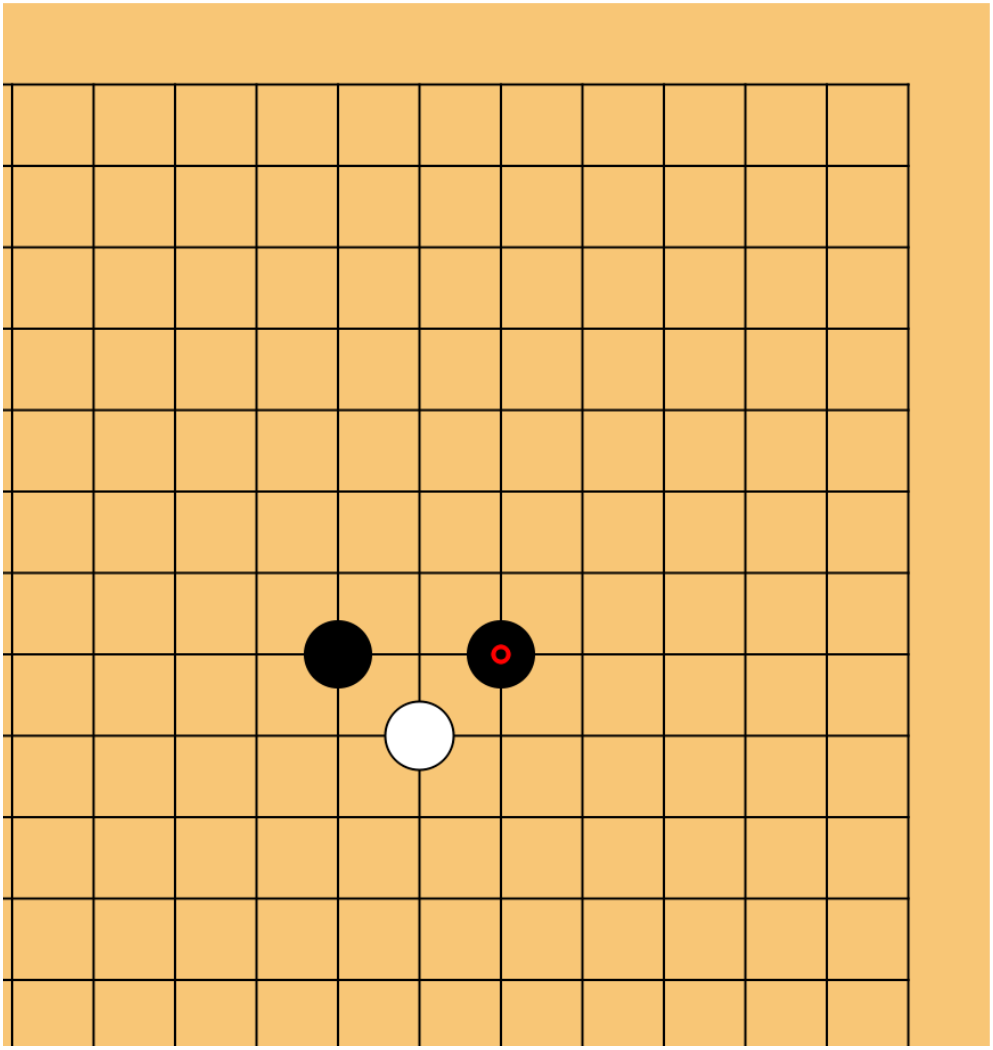


5.2 黑棋长连禁手判负图

- 展示黑棋已经连到 6 子，同时弹出“禁手判负”的提示



5.3 悔棋前后对比图



当前执棋：

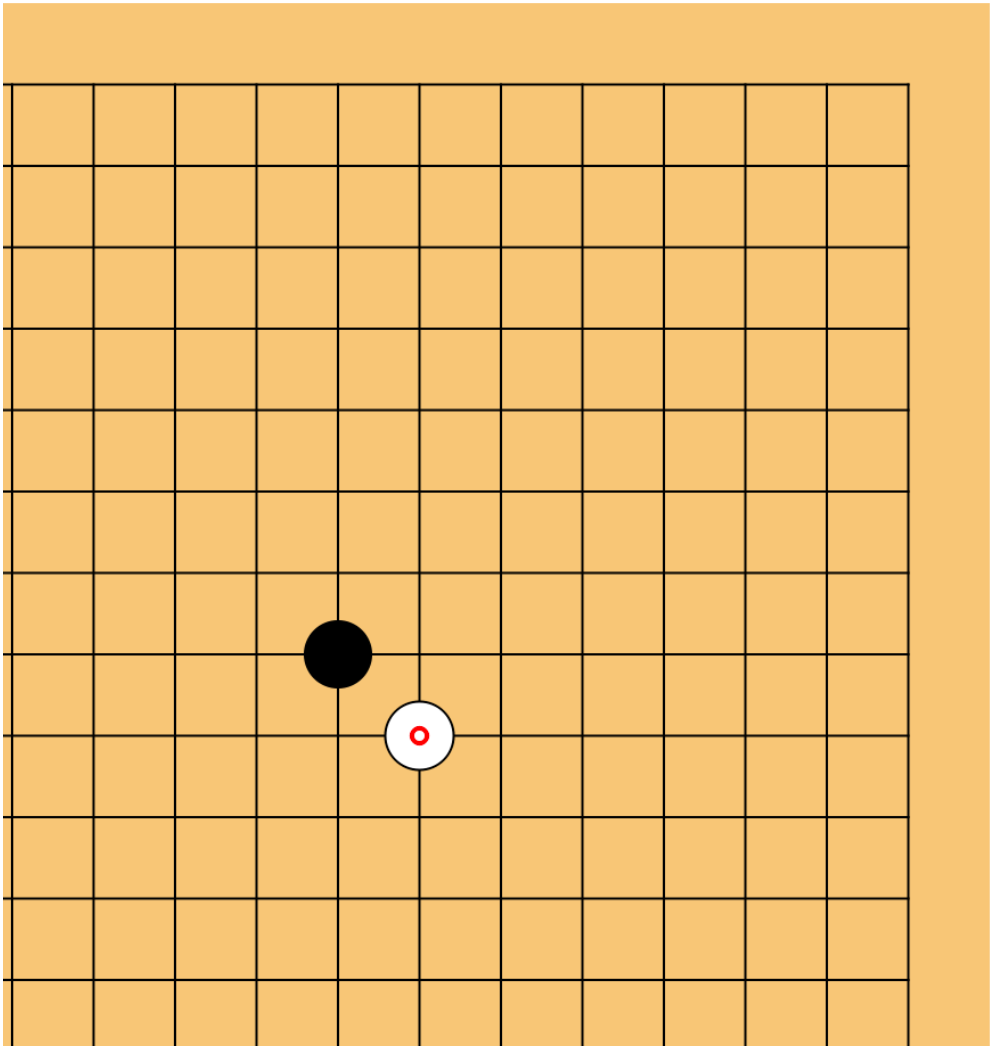
白

黑方剩余：04:47

白方剩余：04:52

状态：进行中

预落子：-



当前执棋：

黑

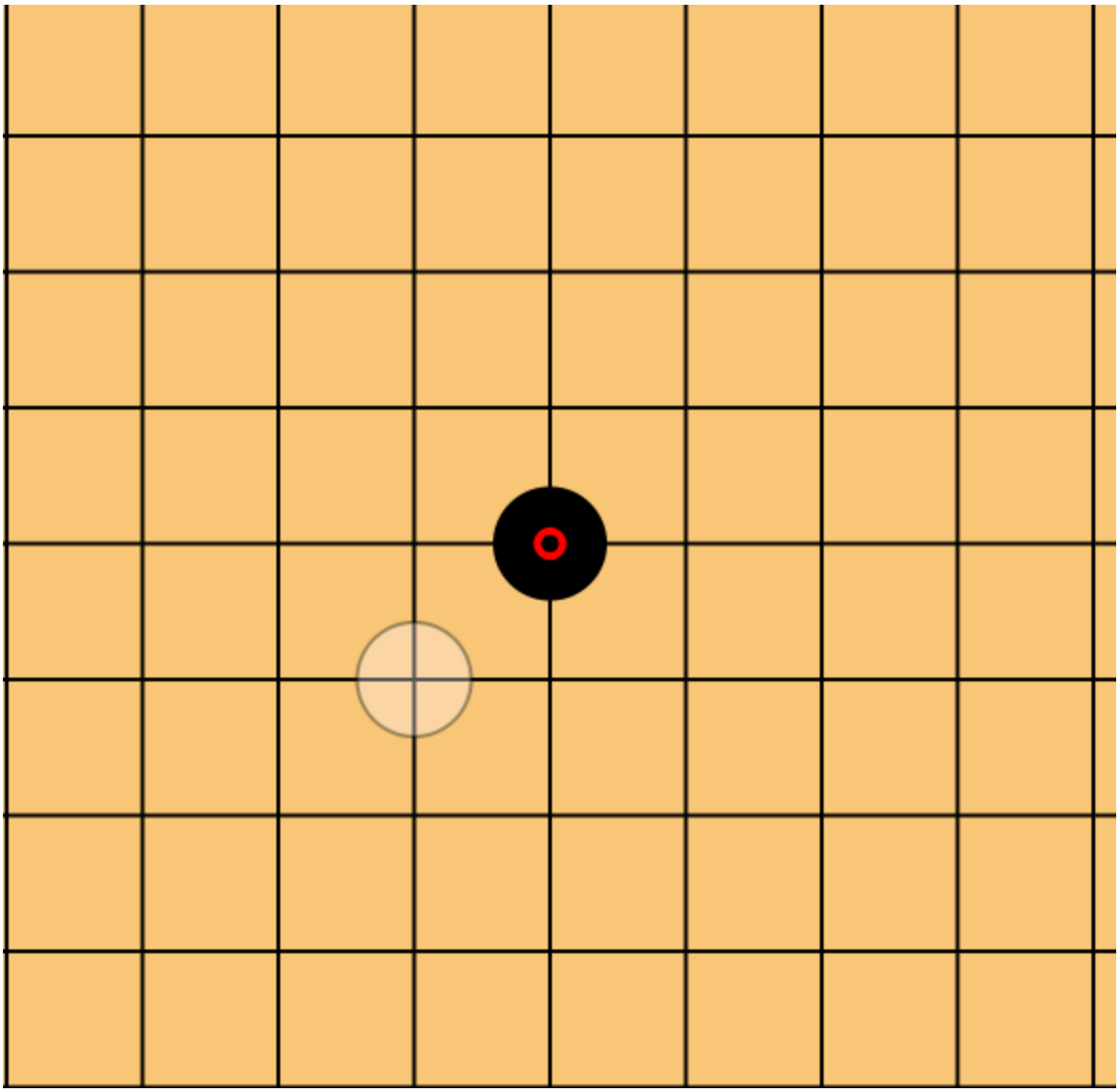
黑方剩余：04:56

白方剩余：04:58

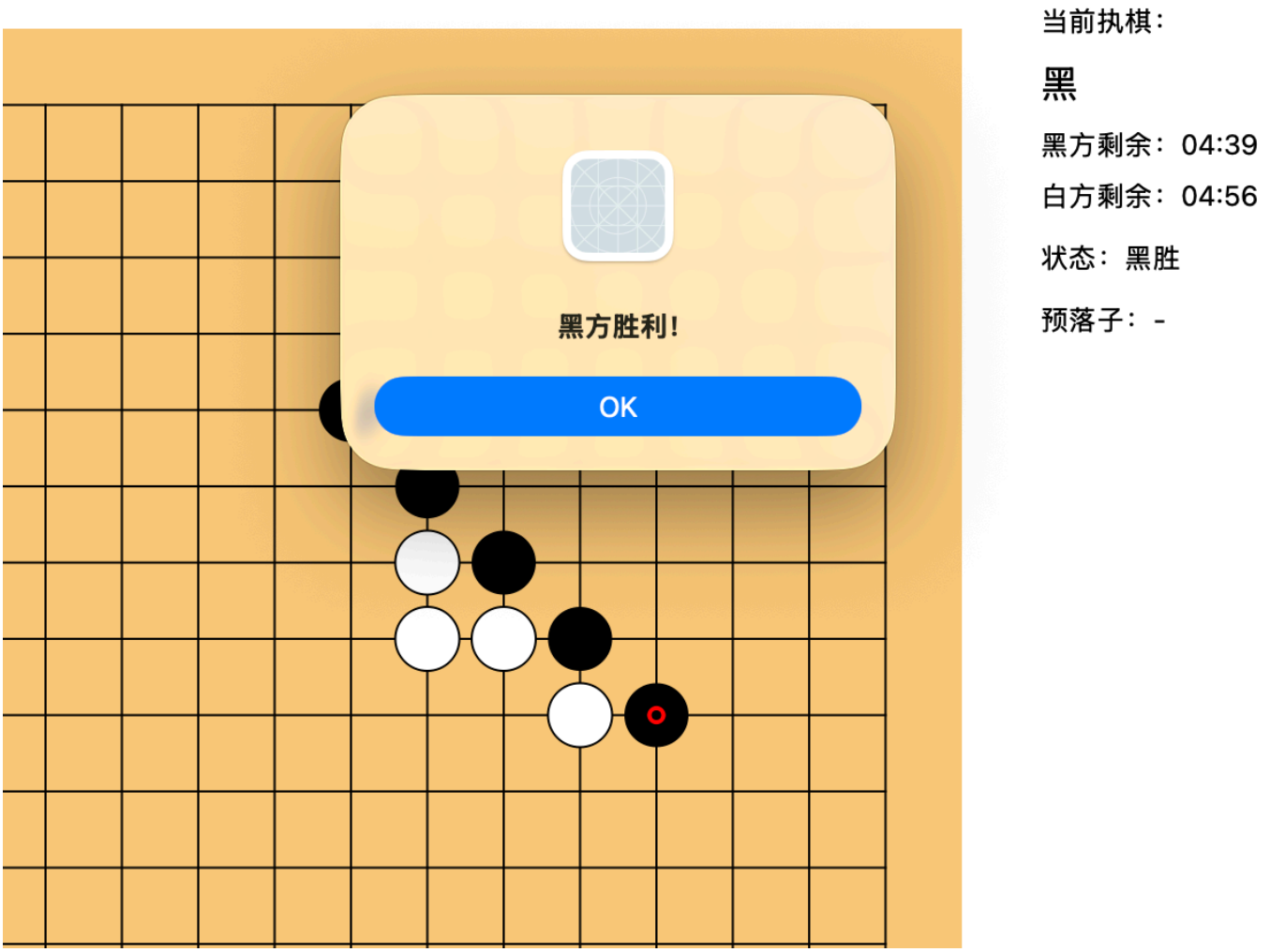
状态：进行中

预落子：-

5.4 落子效果图



5.5胜利弹窗图



6. 总结

这个项目整体上完成了我一开始设定的目标，也是在开发过程中一步一步补充完善起来的。虽然它不是一个特别复杂的大项目，但对于课程作业来说，已经比较完整地体现了界面设计、类的划分、游戏规则判断和实际调试的过程。

当然，这个项目还有可以继续改进的地方，比如棋谱保存、步数记录、完整禁手规则等。不过从这次作业的角度来看，我觉得现在这个版本已经达到了“功能完整、结构清楚、能展示出 C++ 和 Qt 基础能力”的要求。